

IMPORTANTE! Leer atentamente antes de comenzar:

- Crear un proyecto con su Nombre y Apellido.
- Indentar y espaciar correctamente, modularizar, y ser prolijo y ordenado en la codificación (no sumará puntos pero podría restar).
- **ES CONDICIÓN NECESARIA QUE LOS PROYECTOS COMPILEN CORRECTAMENTE E IMPLEMENTEN TODAS LAS FUNCIONALIDADES REQUERIDAS PARA QUE PUEDAN SER CORREGIDOS (NO PUEDE HABER UN MAIN VACIO)** (recomendamos ir verificando si compila bien a medida que van trabajando y entregar sólo lo que compila).
- Al espacio de entrega deberá subirse el proyecto completo comprimido, el cual deberá tener como nombre el apellido y nombre del alumno. Una vez finalizado el horario de entrega, el campus cierra automáticamente la tarea y no pueden subirse entregas ni modificarse las existentes. **REVISAR BIEN EL ARCHIVO SUBIDO, YA QUE UNA VEZ SUBIDA LA ENTREGA Y RETIRADO EL ALUMNO DEL LABORATORIO, LA ENTREGA NO PODRÁ SER MODIFICADA. ES RESPONSABILIDAD DEL ALUMNO LO QUE SUBE Y CÓMO LO SUBE.**

Requerimientos

Te ha contratado la NASA (has llegado lejos en el mundo de la programación! ;)) para que realices un software para la gestión sus misiones de exploración espacial.

1) Diseño del sistema: 60 puntos

Se evaluará no sólo que el sistema esté resuelto, sino también la manera de resolverlo.

Como primer paso, hay que diseñar las clases que sean necesarias para gestionar nuestro sistema, en el cual habrá:

- **Robots exploradores:** poseen un id autoincremental y una cantidad de años de funcionamiento.
- **Astronautas:** poseen un DNI, un nombre y la cantidad de misiones espaciales diferentes en las que participó cada uno.
- **Científicos:** poseen un DNI, un nombre y su antigüedad en la agencia.
- **Personal técnico:** poseen un DNI y un nombre. Poseen como comportamiento específico reparar nave (basta con retornar un String que indique que está reparando la nave).

1. Crear las clases concretas y abstractas que consideres convenientes. Para ello hay que utilizar el paradigma de la POO y aplicar abstracción, encapsulamiento, herencia, modularidad, polimorfismo de métodos, interfaces y responsabilidad del manejo de información.
2. Todas las clases deberán tener los constructores, setter y getter que correspondan. Y en caso de corresponder, también equals(), hashCode() y/o compareTo().
3. Implementar que las clases concretas no puedan tener clases hijas.
4. Implementar el método toString() en la/s clase/s padre/s y sobreescribirlo en la/s hija/s. En todas las clases debe implementarse el método toString() de manera que diga: "Soy un ..." indicando qué tipo de objeto es —robot explorador, astronauta, científico, etc.— y

luego muestre todos sus atributos.

5. Todos los astronautas, científicos y personal técnico trabajan en la promoción de las actividades de la agencia dando charlas en colegios y eventos públicos.
 - Todas las charlas contienen un libreto ya prearmado por la NASA titulado: "Conozca las actividades de la NASA y nuestras misiones espaciales", PERO ADEMÁS:
 - las charlas de los **científicos** suman al libreto genérico el libreto titulado: "Conozca nuestras investigaciones científicas"
 - las charlas del **personal técnico** suman al libreto genérico el libreto titulado: "Conozca nuestra tecnología espacial".
 - Para implementar el método basta con retornar un String con el nombre del libreto/s de cada uno. Pensar qué paradigmas de la POO aplicar en este método.
6. Cada vez que la agencia lanza una Misión de Exploración Espacial, en la misma participan: robots exploradores, astronautas y científicos (el Personal Técnico no). Para ello, todos los participantes deben: realizar la preparación previa para la misión, preparar el equipamiento para la misión, e iniciar la misión. Para implementar cada método basta con retornar un String que indique lo que se está haciendo. Pensar cuál será la manera más adecuada de representar estos comportamientos en el sistema, de manera tal que las clases que deban implementar estos métodos se vean obligadas a hacerlo.

2) Funcionalidades y colecciones: 30 puntos

Se evaluará no sólo que el sistema esté resuelto, sino también la manera de resolverlo.

1. Crear una clase separada llamada GestionDeAgenciaEspacial
2. Dentro de la misma, crear un listado de nombre PersonalAgencia en la cual poder guardar los: Astronautas, Científicos y Personal técnico. El alumno deberá elegir la colección más adecuada, teniendo en cuenta que en la entrevista el cliente nos dijo que:
 - quiere poder acceder a los elementos por su posición en el listado;
 - frecuentemente agregará y eliminará elementos del listado;
 - por lo general, las agregaciones o eliminaciones en el listado no se harán al final sino en el inicio o el medio del listado.
3. En la clase GestionDeAgenciaEspacial deben crearse métodos para manejar el listado de Personal antes mencionado. Por ahora empezaremos creando solamente siguientes métodos:
 - agregar un objeto al listado
 - retornar el objeto del listado que se encuentre en una posición elegida
 - borrar del listado un objeto por su posición
 - mostrar el listado.
4. En la clase GestionDeAgenciaEspacial, crear el método comenzarPreparativosMisionEspacial de manera tal que permita que el mismo reciba cualquier robot explorador, astronauta o científico que pueda realizar una Misión de

Exploración Espacial. No debe permitirse recibir como parámetro de este método personas, robots ni cualquier objeto que NO pueda realizar una Misión de Exploración Espacial. Pero a su vez, el método debe ser flexible para permitir que en el futuro se pueda recibir como parámetro del método personas, robots o cualquier objeto que SI pueda realizar una Misión de Exploración Espacial. En el cuerpo del método se deberán invocar los métodos correspondientes del objeto recibido por parámetro para lograr que dicho objeto comience la preparación previa para la misión y prepare el equipamiento para la misma, y se deberá retornar un String que concatene los dos mensajes resultantes.

3) Implementación del sistema en el Main: 10 puntos (identificar cada ejercicio con una línea de comentario en el Main)

1. Instanciar un objeto de cada una de las clases concretas (no se pide ingreso por teclado de ningún tipo, los objetos a construir deben ser creados con datos "hardcodeados", ingresados por el programador)
2. Invocar los métodos:
 - toString
 - repararNave
 - promocionarAgencia
 - realizar preparacion previa a mision
 - preparar equipamiento para mision
 - iniciar mision *(en cada uno de los objetos creados que los posean, para probar el óptimo funcionamiento de cada clase y cada método).*
3. Agregar al listado de Personal el Astronauta, el Científico y el Personal técnico creados.
4. Mostrar el listado.
5. Mostrar el miembro del personal que se encuentre en una posición elegida del listado (pedir la posición al usuario del sistema).
6. Eliminar un miembro del personal por su posición en el listado (pedir la posición al usuario del sistema) y mostrar el listado para comprobar su eliminación.
7. Invocar el método comenzarPreparativosMision() (basta con pasarle cualquier objeto de los creados en el Main y admitidos como parámetro, no es necesaria la intervención del usuario del sistema)

¡Éxitos a todos en la tarea!